

1 Diffusion và các bước làm cho nó khả thi (Diffusion Models)

PHỤ THUỘC C/17-mo-hình-sinh/01-bon-ho-mo-hình-sinh (vì sao diffusion ổn định hơn GAN); B/09-thiet-ke-ham-mat-mat (hồi quy trên nhiều là bình phương trung bình); C/13-kien-truc-backbone (U-Net và transformer); C/12-co-che-huan-luyen (huấn luyện quy mô lớn).

TÓM TẮT

Diffusion biến một bài toán sinh rất khó thành một chuỗi bài toán khử nhiễu rất dễ, và đó là toàn bộ lý do nó thắng: hàm mất mát của nó là bình phương trung bình trên nhiễu, ổn định như mọi bài hồi quy. Mục này đi từ nguyên lý (quá trình thuận cố định, quá trình ngược học được, liên hệ với score function) tới bốn bước biến nó thành sản phẩm dùng được: bộ lấy mẫu ít bước, dẫn hướng không cần bộ phân lớp, khuếch tán trong không gian ẩn, và cơ chế điều khiển đầu ra. Đọc xong bạn biết mỗi tham số trong một pipeline sinh ảnh đang mua gì bằng gì — số bước, hệ số dẫn hướng, độ mạnh ảnh sang ảnh — và biết *latent diffusion* phá hỏng cái gì khi nó tiết kiệm tính toán. Nếu chỉ nhớ một điều: mọi cơ chế điều khiển trong diffusion đều mua độ trung thành bằng *độ đa dạng*, nên một pipeline chính tới mức đẹp nhất cũng là pipeline có phân phối hẹp nhất — điều nguy hiểm nếu bạn định dùng đầu ra làm dữ liệu huấn luyện.

KIẾN THỨC CẦN CÓ TRƯỚC

- **mô hình sinh (generative model)** — mô hình học phân phối của dữ liệu chứ không học một ánh xạ đầu vào → nhãn; dùng nó là lấy mẫu ra dữ liệu mới, không phải dự đoán.
- **nhiều Gauss (Gaussian noise)** — nhiễu cộng lấy từ phân phối chuẩn; “pha nhiễu” nghĩa là cộng thêm một lượng nhiễu như vậy vào ảnh, làm ảnh mờ dần thành hạt.
- **khử nhiễu (denoising)** — việc ước lượng lại ảnh sạch (hoặc lượng nhiễu đã cộng vào) từ ảnh đã bị pha nhiễu. Diffusion chỉ làm đúng việc này, lặp lại nhiều lần.
- **bộ lấy mẫu (sampler)** — đoạn mã quyết định đi từ nhiễu về ảnh theo mấy bước và mỗi bước dài bao nhiêu. Nó tách rời khỏi mô hình: cùng một mô hình đổi sampler cho ra ảnh khác nhau.
- **không gian ẩn (latent space)** — không gian chiều thấp mà một autoencoder nén ảnh vào; một ảnh 512×512 thành một tensor 64×64 , nên mọi phép tính sau đó rẻ đi hàng chục lần.
- **dẫn hướng (guidance)** — kỹ thuật kéo mẫu sinh ra về phía thoả mãn điều kiện (câu mô tả, ảnh cạnh...) mạnh hơn mức tự nhiên, đổi lại phân phối hẹp hơn.
- **score function (score function)** — gradient của log mật độ xác suất theo dữ liệu, $\nabla_x \log p(x)$; nó là một trường vectơ chỉ về hướng “đi đâu thì hợp lý hơn” tại mỗi điểm trong không gian ảnh.
- **SNR (signal-to-noise ratio)** — tỉ số giữa phần tín hiệu gốc còn lại và phần nhiễu ở một bước; SNR cao nghĩa là ảnh còn gần như nguyên vẹn.

A. Đặt vấn đề

Sau mục 17.1 ta có một yêu cầu rõ ràng: cần chất lượng của GAN mà không có sự bất ổn của một bài toán trò chơi. Ý tưởng của diffusion đơn giản một cách bất ngờ — nếu bài toán “sinh một ảnh từ hư không” quá khó, hãy chẻ nó thành một nghìn bài toán “làm ảnh này bớt nhiễu đi một chút”, mỗi bài dễ tới mức một mạng hồi quy bình thường giải được. Cộng đủ nhiều bước dễ thì ra một bước khó.

Cái phải trả cho sự chẻ nhỏ đó là thời gian: sinh một ảnh cần chạy mạng hàng chục tới hàng nghìn lần. Toàn bộ lịch sử kỹ thuật của mảng này, từ 2020 tới nay, là chuỗi các nỗ lực trả bớt cái giá đó mà không mất chất lượng — và đó cũng là cấu trúc của mục này.

Học mục này thì làm được gì mà trước đó không làm được? Đọc một tệp cấu hình *diffusers* và biết mỗi tham số đang đánh đổi gì; chẩn đoán được ảnh ra xấu là do bộ lấy mẫu, do hệ số dẫn

hướng, hay do bộ mã hoá ẩn; và biết khi nào một mô hình sinh ảnh *không* nên được dùng để tạo dữ liệu huấn luyện.

TRỰC GIÁC

Hãy nghĩ tới một bức ảnh bị nhoè dần trong một chuỗi bản sao chép: bản 1 gần như nguyên vẹn, bản 500 lem nhem, bản 1000 chỉ còn là hạt. Việc đoán bản 1000 từ hư không là bất khả. Nhưng việc đoán bản 499 khi đã có bản 500 thì gần như tầm thường — chỉ phải lùi một nấc. Diffusion là phép nhận ra rằng nếu bạn làm được cái tầm thường đó một nghìn lần, bạn đã làm được cái bất khả. Và điều tinh tế: mấy nấc *cuối* (nhiều nhiều) là nơi mô hình quyết định bức ảnh sẽ vẽ *cái gì*, còn mấy nấc *đầu* (nhiều ít) chỉ tô nốt chi tiết.

B. Quá trình thuận, quá trình ngược

Quá trình thuận không có gì để học: nó chỉ là công thức pha nhiễu Gauss dần dần, và nhờ tính chất cộng của Gauss, ta nhảy thẳng từ ảnh gốc tới bước bất kỳ mà không cần lặp:

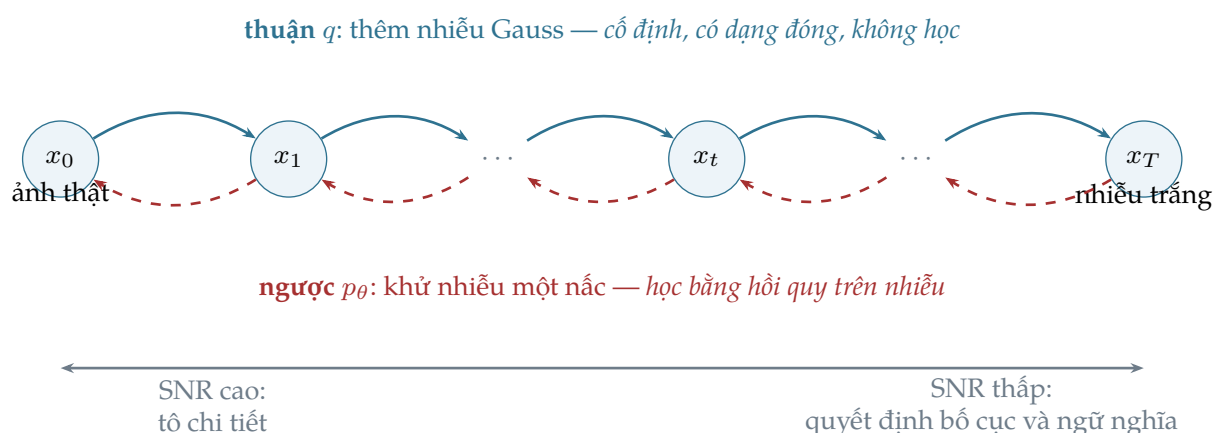
$$q(x_t | x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I), \quad \bar{\alpha}_t = \prod_{s \leq t} (1 - \beta_s).$$

Công thức này nói gì: ở bước t , ảnh còn lại đúng một phần $\sqrt{\bar{\alpha}_t}$ tín hiệu gốc, phần còn lại là nhiễu trắng — và vì có dạng đóng, ta lấy mẫu một t ngẫu nhiên rồi huấn luyện ngay, không phải chạy chuỗi. Đây là lý do kỹ thuật khiến diffusion huấn luyện được ở quy mô lớn.

Tính tay để thấy $T = 1000$ từ đâu ra. Với lịch β tuyến tính từ 10^{-4} tới 0,02, trung bình $\beta \approx 0,0101$, nên $\bar{\alpha}_T \approx \exp(-\sum_t \beta_t) = \exp(-10,1) \approx 4 \times 10^{-5}$, tức $\sqrt{\bar{\alpha}_T} \approx 0,006$. Nói cách khác, sau một nghìn bước thì chưa tới 1% biên độ tín hiệu gốc còn sót lại — vừa đủ để coi x_T là nhiễu trắng thuần túy và bắt đầu quá trình ngược từ đó.

Quá trình ngược mới là phần học. Mạng nhận x_t và t , dự đoán *nhiều* đã thêm vào, và hàm mất mát là bình phương trung bình $\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$ [1]. Đây là điểm mấu chốt của cả họ mô hình: mục tiêu là một bài hồi quy tầm thường, không phải một trò chơi hai người, nên không có bất ổn, không có *mode collapse*, và tăng quy mô thì tốt lên đều đặn.

→ **hệ quả của B/09** — ổn định của diffusion đến thẳng từ dạng của hàm mất mát — bình phương trung bình trên nhiễu — chứ không từ kiến trúc nào.



Hình 1. Hai chiều của cùng một chuỗi. Chiều thuận không có tham số nào, nên toàn bộ khả năng của mô hình dồn vào chiều ngược. Trục dưới giải thích một điều thực hành quan trọng: các bước ở phía nhiễu nhiều quyết định *nội dung* ảnh, nên cắt bớt bước ở phía đó thay đổi ảnh nhiều hơn hẳn cắt bớt ở phía nhiễu ít — đó là lý do các lịch lấy mẫu ít bước không chia đều khoảng cách.

C. Vì sao gọi là “khớp score function”, và vì sao cần nhiều bước

Có một cách nhìn thứ hai cho cùng một mô hình, và nó giải thích được nhiều thứ hơn. Với phân phối p_t của dữ liệu đã pha nhiễu ở mức t , **score function** là $\nabla_x \log p_t(x)$ — vectơ chỉ về hướng “đi đâu thì hợp lý hơn”. Có thể chứng minh rằng mạng dự đoán nhiễu ở trên chính là ước lượng của score function sai khác một hệ số: $\varepsilon_\theta(x_t, t) \approx -\sqrt{1 - \alpha_t} \nabla_x \log p_t(x_t)$ [2]. Nói gọn thì: mô hình học một trường vectơ phủ khắp không gian ảnh, và lấy mẫu là thả một điểm nhiễu vào trường đó rồi để nó trôi về vùng có mật độ cao.

Cách nhìn này trả lời ngay câu hỏi “vì sao cần nhiều bước”: trôi theo một trường vectơ là giải một phương trình vi phân, và mỗi bước rời rạc là một xấp xỉ tuyến tính chỉ đúng trong lân cận nhỏ. Bước dài thì sai số rời rạc hoá lớn, ảnh ra méo. Từ đó suy ra hai hướng cải thiện, và cả hai đều đã thành chuẩn:

- **Đổi bộ giải.** DDIM biến quá trình lấy mẫu ngẫu nhiên thành một phương trình vi phân thường tất định, cho phép nhảy bước dài hơn với cùng mô hình đã huấn luyện [3]. Không huấn luyện lại gì cả — đây thuần tuý là giải số tốt hơn.
- **Làm thẳng quỹ đạo.** *Flow matching* huấn luyện thẳng một trường vận tốc nổi nhiễu với dữ liệu theo đường càng gần đường thẳng càng tốt [4]; quỹ đạo thẳng thì bộ giải cần ít bước hơn. Đây là hướng thiết kế thống trị các mô hình sinh lớn hiện nay.

D. Điều khiển đầu ra: dẫn hướng, không gian ẩn, ràng buộc cấu trúc

I MẠCH 3 — bốn bước biến diffusion thành sản phẩm

1 Vấn đề

Giải pháp

mô hình sinh ra ảnh hợp lý nhưng không theo ý ta.

classifier-free guidance: trong lúc huấn luyện, thỉnh thoảng bỏ điều kiện đi để một mạng duy nhất học cả bản có điều kiện lẫn bản không; lúc lấy mẫu, ngoại suy $\tilde{\varepsilon} = \varepsilon_{\text{vô dk}} + w(\varepsilon_{\text{có dk}} - \varepsilon_{\text{vô dk}})$ [5]. Công thức này nói gì: lấy hiệu giữa “có điều kiện” và “không điều kiện” làm hướng của điều kiện, rồi đi quá lên w lần theo hướng đó.

Giá phải trả

$w > 1$ không phải là “làm đúng hơn” mà là *làm phân phối hẹp lại*: bám prompt tốt hơn, đa dạng kém đi, và ở w lớn thì màu bị cháy.

2 Vấn đề

Giải pháp

chạy khuếch tán trên không gian pixel quá đắt.

nén ảnh bằng một autoencoder rồi khuếch tán trong không gian ẩn [6]. Với hệ số nén 8, ảnh 512×512 còn 64×64 vị trí — ít hơn 64 lần, và vì chi phí **attention** tăng theo bình phương số vị trí, phần đó rẻ đi hàng nghìn lần.

Hệ quả

đây chính là bước biến diffusion từ nghiên cứu thành sản phẩm chạy được trên GPU tiêu dùng.

↪ **ràng buộc bởi** B/11 — lợi ích của không gian ẩn tính trực tiếp từ ngân sách bộ nhớ và chi phí attention theo số vị trí.

Giá phải trả

chất lượng bị chặn trên bởi chất lượng autoencoder; chi tiết rất nhỏ — chữ nhỏ, mắt người ở xa, hoa văn mịn — bị phá ngay ở khâu nén và không mô hình khuếch tán nào cứu lại được.

3 Vấn đề

Giải pháp

U-Net khó mở rộng quy mô một cách có quy luật.

thay bằng **transformer** thao tác trên các **token** của không gian ẩn — **diffusion transformer** [7].

Hệ quả

quy luật mở rộng quy mô trở nên dự đoán được, giống như ở mô hình ngôn ngữ.

4 Vấn đề

Giải pháp

prompt bằng chữ không đủ để ràng buộc cấu trúc hình học.

ControlNet: nhân bản phần mã hoá của mô hình gốc, cho nhánh sao chép nhận điều kiện cấu trúc (cạnh, tư thế người, bản đồ độ sâu), và nối nó vào mô hình gốc qua các tầng tích chập *khởi tạo bằng 0* [8]. *Chi tiết khởi tạo 0 mới là chỗ hay*: ở bước huấn luyện đầu tiên nhánh mới đóng góp đúng bằng không, nên mô hình gốc không bị phá; huấn luyện chỉ *thêm* khả năng chứ không làm hỏng prior đã có.

Giá phải trả

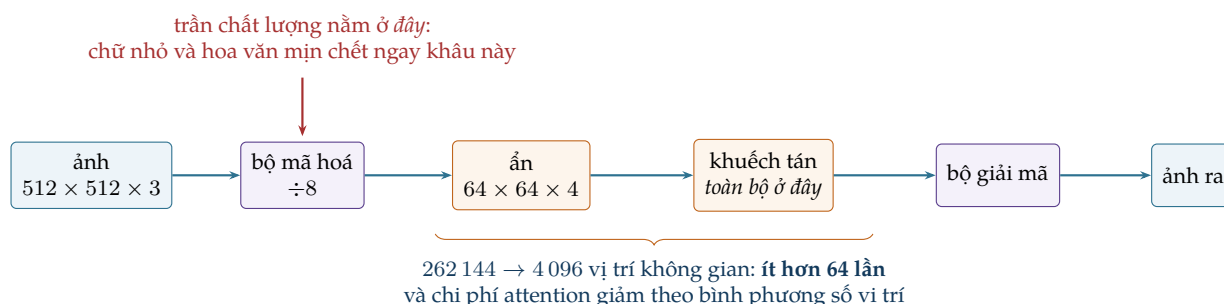
càng nhiều ràng buộc thì càng ít đa dạng — cùng một quy luật với dẫn hướng ở bước 1.

Bảng 1. Ba cách điều khiển đầu ra. Cả ba mua độ trung thành bằng cùng một đồng tiền là độ đa dạng.

Cách	Cơ chế	Ràng buộc được gì	Mất gì
Dẫn hướng (CFG)	Ngoại suy giữa nhánh có và không điều kiện	Ngữ nghĩa, độ bám prompt	Đa dạng; w lớn gây cháy màu
ControlNet	Nhánh sao chép, nối bằng tầng khởi tạo 0	Cấu trúc hình học: cạnh, tư thế, độ sâu	Tự do bố cục; thêm chi phí tính toán
Ảnh sang ảnh / inpainting	Bắt đầu từ ảnh đã pha nhiễu tới mức $t < T$	Bố cục và màu của ảnh gốc	Mức thay đổi bị chặn bởi t đã chọn

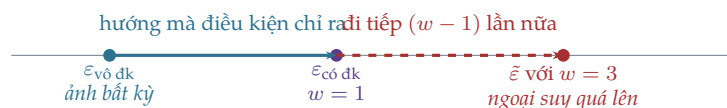
Riêng dòng cuối bảng đáng một câu giải thích, vì nó là hệ quả trực tiếp của Hình 1: tham số “độ mạnh” trong ảnh sang ảnh thực chất chỉ là *chọn bắt đầu từ nấc nào*. Bắt đầu ở nấc nhiều ít thì ảnh gốc còn nguyên và mô hình chỉ tô lại bề mặt; bắt đầu ở nấc nhiều thì bố cục cũng bị viết lại. Không có gì bí ẩn ở đây — chỉ là một chỉ số trên trục thời gian.

Hình 2 và Hình 3 vẽ hai bước quan trọng nhất trong mạch trên.



Hình 2. Latent diffusion, và chỗ nó lấy tiền về cũng như chỗ nó trả tiền ra. Toàn bộ phần dất — hàng chục tới hàng trăm lượt chạy mạng — được dời vào không gian ẩn nhỏ hơn 64 lần về số vị trí, và đó là lý do một mô hình sinh ảnh chạy được trên GPU tiêu dùng. Cái giá nằm ở mũi tên đỏ: mọi chi tiết mà bộ mã hoá không giữ được thì không mô hình khuếch tán nào ở phía sau dựng lại được. Vì vậy phép chẩn đoán đầu tiên khi ảnh mất chi tiết là mã hoá rồi giải mã ngay ảnh gốc, bỏ qua khuếch tán.

càng sang phải: bám prompt càng chặt, phân phối càng **hẹp**, màu càng dễ cháy



Hình 3. Dẫn hướng không cần bộ phân lớp, nhìn như một phép ngoại suy trên đường thẳng. Hiệu giữa dự đoán có điều kiện và dự đoán không điều kiện chính là “hướng của điều kiện”; hệ số w quyết định đi bao xa theo hướng đó. $w = 1$ là dừng đúng dự đoán có điều kiện; $w > 1$ là đi *quá* lên. Điều quan trọng cần đọc ra từ hình: không có chỗ nào trong phép này làm mô hình “đúng hơn” — nó chỉ đẩy mẫu về vùng điển hình và cắt bớt đuôi, nên ảnh riêng lẻ đẹp hơn còn tập ảnh thì nghèo đi.

E. Giới hạn và trường hợp hỏng

- **Chậm là tính chất, không phải lỗi.** Mọi cách tăng tốc — ít bước, chưng cất bộ lấy mẫu, quỹ đạo thẳng — đều đang xấp xỉ cùng một quỹ đạo, và ở số bước rất nhỏ thì chất lượng và đa dạng đều tụt. Chưng cất bộ lấy mẫu là đúng mẫu hình “dời chi phí sang lúc huấn luyện” của mục 17.2.
 ↪ cùng nguyên lý với D/20 — cùng phép đổi chi phí suy luận lấy chi phí huấn luyện đã gặp ở chưng cất mô hình.
- **Trần chất lượng nằm ở autoencoder.** Nếu ảnh ra mất chữ nhỏ hoặc chi tiết mịn, đừng đổ cho mô hình khuếch tán trước khi kiểm tra: mã hoá rồi giải mã ngay ảnh gốc, không chạy khuếch tán, và xem chi tiết đó có sống sót không. Phép thử này mất ba mươi giây và loại bỏ được một nửa số giả thuyết.
- **Đếm, chữ, và quan hệ không gian vẫn yếu.** Đây là hệ quả của việc điều kiện đi vào mô hình qua *cross-attention* trên biểu diễn văn bản, chứ không qua một cơ chế suy luận rời rạc nào.
- **Lệch giữa mô hình và bộ lấy mẫu.** Một mô hình huấn luyện với lịch nhiều này, chạy bằng bộ lấy mẫu giả định lịch khác, cho ra ảnh xấu mà không có thông báo lỗi nào. Đây là lỗi cấu hình phổ biến nhất trong thực hành.

CẠM BẮY

Tăng hệ số dẫn hướng cho tới khi ảnh “đẹp nhất” là một cái bẫy có hậu quả xa. Dẫn hướng mạnh không làm mô hình đúng hơn — nó *cắt bớt đuôi phân phối*, giữ lại phần lõi điển hình nhất. Ảnh riêng lẻ đẹp hơn, nhưng tập ảnh sinh ra nghèo hơn hẳn tập ảnh thật. Nếu bạn định dùng đầu ra làm dữ liệu huấn luyện, đây chính là cơ chế gây thu hẹp phân phối qua từng vòng lặp: mỗi thế hệ học từ đuôi đã bị cắt của thế hệ trước. Xem C/17-mo-hình-sinh/04-low-level-vision.

↪ **tiền đề** C/17/04 — muốn hiểu vì sao dữ liệu sinh có thể đầu độc mô hình sau, phải hiểu trước rằng mọi nóm điều khiển ở đây đều cắt đuôi phân phối.

CẦN NHỚ

Diffusion thắng không phải vì ý tưởng đẹp hơn GAN mà vì nó biến bài toán sinh thành một bài *hồi quy*, và hồi quy thì mở rộng quy mô được. Ba con số đáng nhớ khi chỉnh một pipeline: số bước quyết định sai số rời rạc hoá, hệ số dẫn hướng quyết định đánh đổi trung thành ↔ đa dạng, và hệ số nén của autoencoder quyết định trần chi tiết. Ba tham số đó độc lập nhau, và chẩn đoán sai lầm hay gặp nhất là đổ lỗi cho tham số thứ nhất khi thủ phạm là tham số thứ ba. [T2]

F. Kết nối

Mục này là phần chi tiết của họ thứ tư trong C/17-mo-hinh-sinh/01-bon-ho-mo-hinh-sinh; “chưng cất bộ lấy mẫu” là bản sao của mẫu hình ở C/17-mo-hinh-sinh/02-toi-uu-tren-pixel và ở D/20-toi-uu-va-nen-mo-hinh. Việc dùng đầu ra làm dữ liệu huấn luyện và hậu quả thu hẹp phân phối nằm ở C/17-mo-hinh-sinh/04-low-level-vision; phần kiến trúc transformer thay U-Net nối về C/13-kien-truc-backbone/03-vit-va-inductive-bias; chi phí bộ nhớ và tính toán của attention theo số vị trí nằm ở B/11-gioi-han-tinh-toan.

■ Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao quá trình thuận *không có tham số* nào lại là điều kiện kỹ thuật khiến diffusion huấn luyện được ở quy mô lớn?
2. Cách nhìn “score function” giải thích được điều gì mà cách nhìn “khử nhiễu từng nấc” không giải thích được?
3. Latent diffusion tiết kiệm tính toán bằng cách nào, và bạn thiết kế phép thử ba mươi giây nào để biết chi tiết bị mất là do khâu nén hay do khâu khuếch tán?
4. Vì sao tăng hệ số dẫn hướng làm ảnh đẹp hơn nhưng làm tập ảnh nghèo hơn — và khi nào điều đó là nguy hiểm chứ không chỉ là đáng tiếc?
5. Tầng tích chập nổi của ControlNet được khởi tạo bằng 0. Bỏ chi tiết đó đi thì hỏng ở đâu, trong bước huấn luyện thứ mấy?
6. Theo Hình 1, nếu chỉ được giữ 10 trong 1000 bước, bạn đặt chúng ở đâu và vì sao?

Nguồn

- [1] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *NeurIPS*, 2020.
- [2] Y. Song, J. Sohl-Dickstein, D. P. Kingma, et al., “Score-Based Generative Modeling Through Stochastic Differential Equations,” in *ICLR*, 2021.
- [3] J. Song, C. Meng, and S. Ermon, “Denoising Diffusion Implicit Models,” in *ICLR*, 2021.
- [4] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, et al., “Flow Matching for Generative Modeling,” in *ICLR*, 2023.
- [5] J. Ho and T. Salimans, “Classifier-Free Diffusion Guidance,” *arXiv:2207.12598*, 2022.
- [6] R. Rombach, A. Blattmann, D. Lorenz, et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” in *CVPR*, 2022.
- [7] W. Peebles and S. Xie, “Scalable Diffusion Models with Transformers,” in *ICCV*, 2023.
- [8] L. Zhang, A. Rao, and M. Agrawala, “Adding Conditional Control to Text-to-Image Diffusion Models,” in *ICCV*, 2023.